Temple University
Japan Campus

Group Report

MakeMe

Team: Karl Fernandez
Moe Odaki
Scout Peterson

Github: https://github.com/kidevil1290/makeme

Date: November 12, 2026

Contents

1 Abstract **2**

2 Introduction **2**
2.1 Background and Context 2
2.2 Motivation and Problem Statement 3
2.3 Project Goals and Objectives 3

3 Literature Review and Related Work **4**
3.1 Relevant Research and Existing Products 4
3.2 Comparison With the Proposed Project 5

4 Project Requirements **5**
4.1 Product Vision and Scope 5
4.2 High-Level Requirements 5
4.3 Target Users and Personas 6

5 System Design **6**
5.1 System Architecture 6
5.2 Use Case Diagram 6
5.3 UML Class Diagram 7
5.4 Sequence Diagram 9
5.5 Design Patterns and Justification for Their Use 9

6 Implementation and Proof of Concept **10**
6.1 Tools and Technologies 10
6.2 Implementation of the Main Features 10
6.2.1 Front-End Screens and Navigation 10
6.2.2 Avatar System 11

7 Required Resources and Feasibility **12**
7.1 Hardware, Software, and Other Resources 12
7.2 Feasibility Within the Available Time and Resources 12

8 Results and Discussion **13**
8.1 Proposed Features and Their Completion Status 13
8.2 Testing and Results 13
8.3 Challenges, Limitations, and Reasons for Incomplete Features 13

9 Conclusion and Future Work **13**
9.1 Summary of Achievements 13
9.2 Possible Improvements and Extensions 13

10 References **14**

1. Abstract

Maintaining consistent exercise habits over an extended period can be challenging, particularly when fitness progress is primarily represented through numerical data. MakeMe is a gamified fitness-tracking application designed to address this challenge by integrating personalized avatar progression into the fitness-tracking experience. The application translates users' real-world activities into visual changes in a personalized virtual character, with its initial minimum viable product (MVP) incorporating four core components: character customization, workout logging, muscle-specific progression, and a 90-day game cycle. Character customization allows users to personalize their avatars, establishing a stronger sense of connection with their virtual representations, while the 90-day cycle introduces a structured progression period that provides a longer-term objective. At the core of this system, workout logging records users' training activity, which drives muscle-specific growth in their avatars. This creates a direct relationship between real-world exercise and avatar development, allowing users to visualize how their individual training activities contribute to their avatars progression. Through these interconnected mechanics, MakeMe aims to provide users with a more engaging representation of their fitness progress and encourage consistent exercise through game-inspired progression mechanics.

2. Introduction

2.1. Background and Context

Conventional fitness-tracking applications provide users with a range of tools to monitor and manage their physical activities. Activities that were traditionally recorded through handwritten logs and manual calculations can now be conveniently organized within digital platforms, allowing users to track workout frequency, monitor performance, and review their progress over time. These features provide valuable feedback and help users establish a structured approach to their fitness routines. Across many of these applications, numerical statistics remain a central method of presenting users' progress. However, while these measurements provide useful information, they offer limited opportunities for users to visually identify with their progress or develop an aspirational connection with their fitness goals. As a result, the ability to monitor fitness activity does not necessarily translate into sustained motivation or consistent exercise habits.

One potential approach to making fitness progress more engaging is through the use of personalized avatars. These avatars can serve as visual representations of users, gradually developing alongside their real-world physical activities. By allowing users to customize their avatars, applications can create opportunities for stronger self-identification, making the avatar's development more personally meaningful. Furthermore, allowing these characters to develop toward an idolized physique introduces an aspirational element, giving users a visual representation of what they may hope to achieve through their own fitness journey. Through this combination of self-representation and aspiration, avatars can provide users with a more personal and engaging connection to their fitness goals.

Building upon these concepts, MakeMe is a gamified fitness-tracking application designed to connect users' real-world exercise activities with the development of a personalized avatar. The initial minimum viable product (MVP) focuses on four core components: character customization, workout logging, muscle-specific progression, and a 90-day game cycle. Users can personalize their avatars and record their workout activities, which contribute to the gradual development of specific muscle groups within the avatar. Rather than representing physical development as a generalized progression, MakeMe aims to reflect users' individual training habits through muscle-specific visual changes. This progression is further structured around a 90-day cycle,

providing users with a defined period in which to develop their avatars and work toward their fitness goals. By combining workout tracking, character customization, and visual progression, MakeMe seeks to create a more personalized and game-inspired approach to monitoring fitness development.

2.2. Motivation and Problem Statement

Despite the availability of fitness-tracking applications, maintaining consistent engagement with exercise routines remains a challenge for many individuals. Recording workouts and monitoring physical activity can help users organize their fitness routines, but these activities alone do not necessarily provide a compelling reason to remain engaged over an extended period. There is an opportunity to explore how game-inspired progression systems can make continued participation more meaningful by connecting users' exercise efforts with a sense of personal development. The challenge addressed by MakeMe is to design an application that translates individual workout activities into personalized, muscle-specific avatar progression while providing a structured experience through its 90-day game cycle.

The motivation behind developing MakeMe stems from the opportunity to make fitness tracking more engaging through concepts commonly found in video games. By incorporating personalized character development into the fitness-tracking experience, the project explores how users can establish a stronger connection with their fitness goals through virtual self-representation and aspirational progression. Rather than focusing exclusively on recording physical activities, MakeMe aims to provide users with a visual representation of their training efforts, where individual workouts contribute to the gradual development of a character they have personalized. This approach seeks to make fitness progression more personally meaningful while introducing game-inspired mechanics that may encourage users to remain engaged with their exercise routines. Ultimately, the project aims to explore how principles of character customization and progression can be applied to fitness applications to support a more engaging approach to exercise tracking.

2.3. Project Goals and Objectives

MakeMe's primary goal is to explore how conventional fitness-tracking functionality can be extended through game-inspired character progression. By incorporating personalized virtual representations into the fitness-tracking experience, the project aims to establish a more engaging approach to visualizing users' fitness development. The initial MVP serves as the foundation for this broader vision, establishing the essential functionality and progression systems upon which additional features can be developed and integrated in the future.

To establish this foundation, the project pursues the following objectives.

1. Develop a Personalized Avatar System: Implement a customizable virtual character whose appearance can be personalized by users with individual muscle groups that develops independently through recorded workout activities.
2. Integrate Core Fitness-Tracking Functionality: Implement essential workout-logging functionality that allows users to record their exercise activities serving as the foundation for the application's avatar progression system.
3. Establish a Structured Progression System: Develop a 90-day progression cycle that provides users with a defined period to track their activities, observe avatar development, and work toward longer-term fitness goals.

3. Literature Review and Related Work

3.1. Relevant Research and Existing Products

MakeMe's avatar system draws on virtual self-modeling through vicarious reinforcement by Fox and Bailenson, which explored how observing virtual representation of oneself receiving reinforcement would influence exercise behavior [1]. However, their research used virtual representation that resembled the participants, whereas MakeMe features fantasy-based character development that would later skew away from the users physical appearance. This distinction is intentional, as MakeMe allows users to watch their avatars develop into increasingly muscular and powerful characters, potentially inspiring them to aspire to the strength and appearance of their virtual counterparts, motivating them to maintain healthy habits. This design aligns the concept of wishful identification mentioned in the review research by Praetorius and Gorlich on the Proteus Effect, which describes how an avatar's characteristics may influence a users behavior and self-perception [2].Drawing on these concepts, MakeMe's avatar system is designed around two complementary forms of progression: reflective and aspirational. Where the development of the avatar is a direct reflection of users actions, while aspirational progression allows the avatar to evolve towards an idolized physique that users may wish to achieve themselves. While existing research does not directly establish a link between avatar progression and the development of healthy habits, incorporating them may still offer motivational benefits compared with health tracking applications without an avatar.

Although ranking of the top 10 fitness trackers may vary due to subjective reviews and potential biases, many of them can be broadly classified into three categories based on their approaches to fitness tracking and gamification.

Category 1 consists of conventional fitness trackers that incorporate little to no gamification, focusing primarily on workouts, activity logging, and social features [3], [4]. Their popularity suggests that stand-alone fitness monitoring remains an established approach to helping users track their progress and maintain their fitness routine. However, this category represents only one approach to fitness tracking and does not establish whether users would benefit from engaging or game-oriented alternatives.

Category 2 consists of challenge-based gamified fitness trackers that are oriented to encourage physical activity through step-count goals, competitions, social challenges, and real-world incentives. In 2020, (Neupane et al.) reviewed 103 gamified fitness tracker apps and found that challenges, competition and social influence frequently appeared together, while social features played a central role across nearly every identified game genre [5]. These findings demonstrate how conventional activity tracking can be enhanced with game mechanics that encourage users to remain active through external rewards, social interaction, and competition. However, this approach primarily builds on existing fitness tracking activities rather than making exercise part of a broader, evolving game experience.

Category 3 consists of stand-alone games that integrate physical activity directly into gameplay, encouraging users to exercise as they pursue in-game objectives. Games such as PokemonGO and Wii Sports demonstrate the popularity of this approach making physical activity more engaging through exploration and interactive gameplay [6], [7]. In these games, physical activity is not simply a means of tracking progress but an integral part of the game itself. Players are motivated to move, explore, and engage with their environment to achieve in-game goals. However, research on PokemonGO suggests that increases in physical activity may be short-lived, highlighting the challenge of sustaining user engagement and long-term exercise habits.

3.2. Comparison With the Proposed Project

Initially, MakeMe was envisioned as an application that would combine elements from the three approaches discussed above, bringing together conventional fitness tracking, gamification, and interactive gameplay. However, as the project developed, its scope was refined to focus on a more specific aspect of the fitness-tracking experience: connecting users' workout activities with the development of their avatars. Rather than attempting to incorporate the various challenges, social features, and gameplay mechanics found in existing solutions, the current MVP places its emphasis on workout logging and muscle-specific avatar progression. Through this approach, recorded exercise activities directly contribute to the development of corresponding muscle groups within a personalized character. This shift allows MakeMe to concentrate on its central design concept providing users with a visual representation of their training efforts through an avatar that develops alongside their fitness journey.

4. Project Requirements

4.1. Product Vision and Scope

MakeMe is a gamified fitness-tracking application that aims to connect users' real-world fitness activities to a developing personalized avatar. The application is designed around the idea that progress from workouts can be represented through a character that gradually develops alongside the user's training habits. Rather than relying exclusively on numerical workout records, MakeMe complements them with visual character development to provide users with another way to observe their fitness progress.

The MVP focuses on four main components: character customization, workout logging, muscle-specific avatar progression, and a 90-day progression cycle. Users can personalize their avatar upon creation before recording their workouts. Based on the recorded workouts, the avatars corresponding muscle groups gradually develops, allowing different areas of the character to progress according to the user's training habits. The 90-day cycle provides a structured period for users to work toward their fitness goals, with the opportunity to ascend into a new cycle, receiving badges to recognize their accomplishments.

The original conceived idea was a broader system that considered nutrition, hydration, and sleep tracking, alongside additional mechanics such as avatar decay, recovery, and bulk progression. However, these features have been excluded from the initial MVP due to the interest of time and the amount of resources they required. Several of the proposed systems also depended on extensive visual assets and additional implementation, which would significantly increase the project's development complexity. Ultimately, MakeMe concluded to prioritize the core workout and avatar progression mechanics before considering integrating additional features. This narrower scope allowed the project to concentrate on its primary mechanic while establishing a foundation for future expansion.

4.2. High-Level Requirements

- **R1.** Initial Avatar Set Up. The system will allow users to personalize their avatars head at creation and a short survey will be used to establish the avatars initial baseline.
- **R2.** Workout Logging. A dedicated workout logger will collect duration, select muscle group, and intensity level from user input.
- **R3.** Muscle-Specific Growth. The system will take the logged data from workout logger and calculate and apply growth to the corresponding muscles.

- **R4.** Avatar Display. The system will display the avatar's current appearance and muscle development
- **R5.** 90-day Cycle. The system will track user progression within 90-day cycle.
- **R6.** Ascension System. The system will allow users to ascend into a new progression cycle upon completion of the current cycle.
- **R7.** Reward System. The system will reward users with a badge after ascension.

4.3. Target Users and Personas

MakeMe primarily targets individuals who want to develop healthier habits through an interactive and gamified fitness application. The following personas represent two potential user groups.

Persona 1: Alex, 21 — University Student

- **Background:** A university student who enjoys video games but struggles to maintain a consistent exercise routine.
- **Technical Experience:** Familiar with smartphones, mobile applications, and video games, but has limited experience with fitness tracking applications.
- **Goal:** Develop a regular workout routine and stay motivated by tracking personal progress.
- **Relevance to MakeMe:** Interested in using avatar progression and visual rewards to make fitness tracking more engaging and enjoyable.

Persona 2: Sarah, 29 — Office Worker

- **Background:** An office worker who exercises several times per week but finds it difficult to maintain healthy habits due to a busy schedule.
- **Technical Experience:** Comfortable using smartphones and fitness applications to track workouts and daily activities.
- **Goal:** Manage workout routines, nutrition, hydration, and sleep habits in one application.
- **Relevance to MakeMe:** Interested in combining multiple health tracking features with a visual avatar that reflects daily habits and long-term consistency.

5. System Design

5.1. System Architecture

The application is developed using Flutter and Dart, with Provider for state management and coordinating UI updates. For development and testing, Android Studio provides an integrated environment in which the application can be deployed and run through an Android emulator, allowing the team to evaluate its functionality and visual presentation. Its architecture emphasizes modularity and separation of responsibilities, utilizing immutable data models and a layered sprite rendering system to support customizable avatars and muscle-specific visual progression.

5.2. Use Case Diagram

The primary actor is the User, with a one-time avatar creation interaction. Its main interaction is recording workouts through the workout logger, managing workout templates and viewing

the avatar's visual development as it changes in stature based on the users recorded activity represented in figure 1

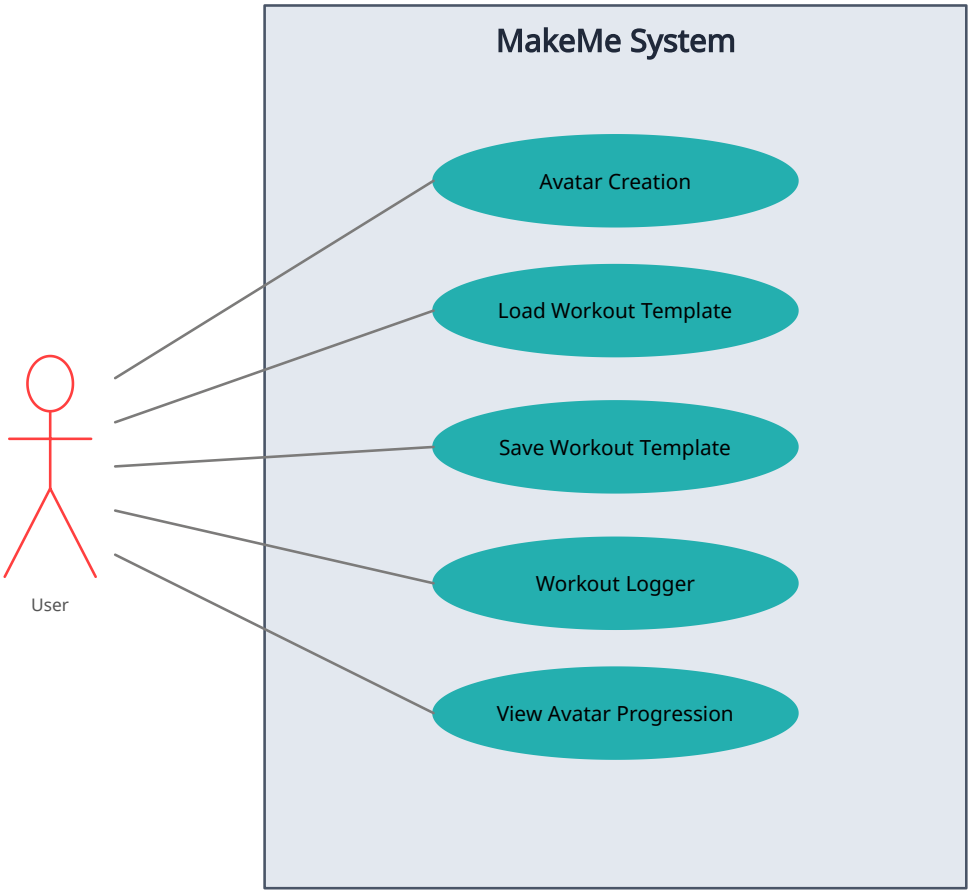


Figure 1: Use-case diagram for MakeMe.

5.3. UML Class Diagram

MakeMe's class diagram is represented in figure 2. The classes fall into four groups: user and avatar data, workout logging, cycle management and screen navigation, with a set of enumerations defining the fixed options used across application.

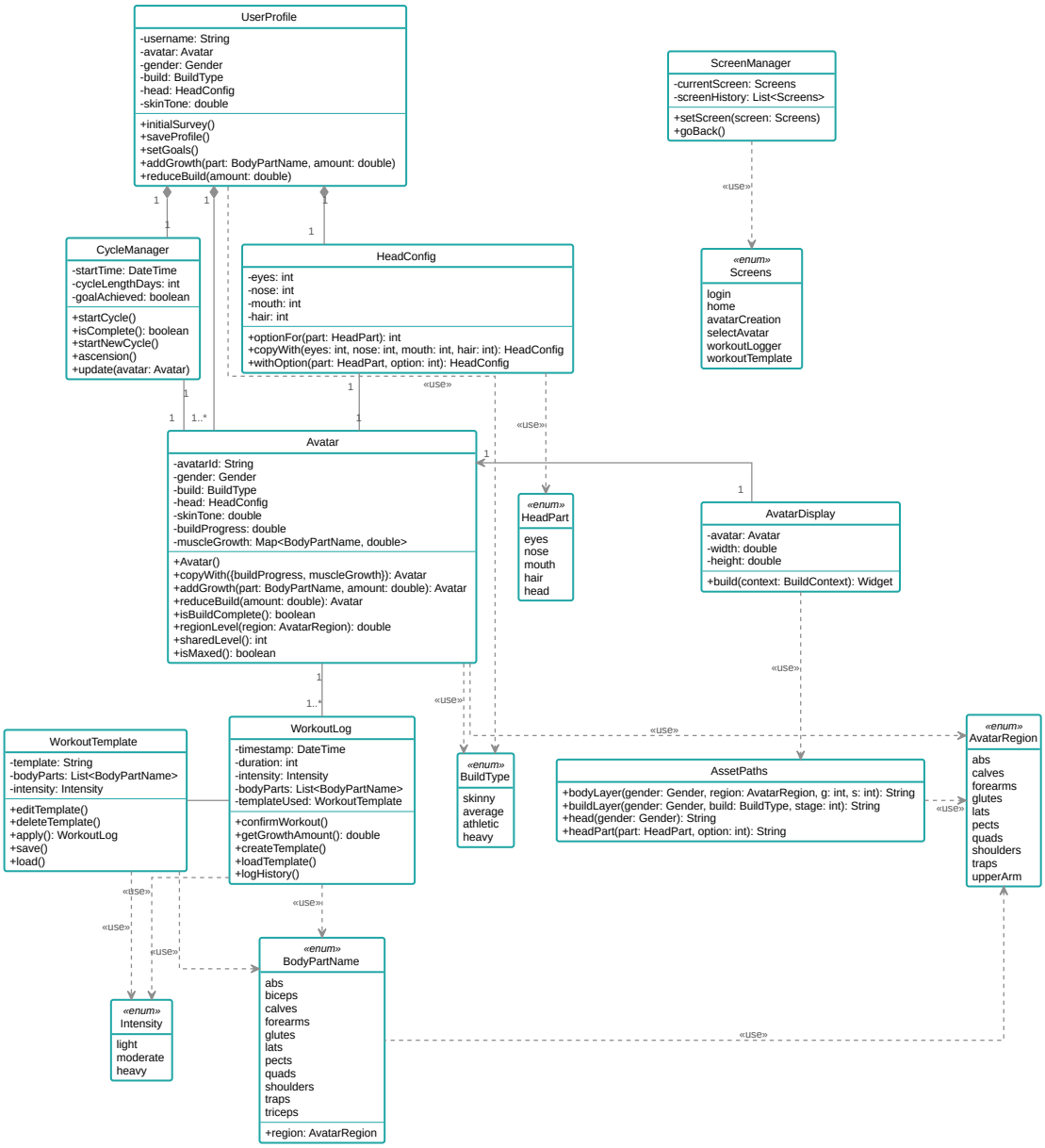


Figure 2: UML class diagram for MakeMe.

The UserProfile class is at the center of this design. It owns the user's avatar and is related to one or more avatars, so earlier avatars can be kept as a history once a cycle ends. The avatar class is immutable , so methods such as copyWith() and addGrowth() return a new Avatar instead of changing the existing one, which allows the interface to detect a change and redraw the avatar. Each avatar has many WorkoutLog entries, and each log determines how much growth a session contributes to the muscle groups trained. The CycleManager class tracks the 90-day cycle, and the ScreenManager class handles navigation between screens. The AvatarDisplay widget draws the current Avatar by stacking sprite layers whose file paths are built by AssetPaths. Workout logging records each muscle separately through BodyPartName, including the biceps and triceps, while each body part maps to one of the ten visual regions in AvatarRegion. As a result, the biceps and triceps both contribute to the single upper-arm layer. The Avatar also stores a buildProgress value for the skinny and heavy starting builds, and its sharedLevel() method calculates the shared muscle level used to select each region's sprite, as described in the Avatar System section.

5.4. Sequence Diagram

The sequence diagram in figure 3 showcases the central interactions that occurs when a user records a workout.

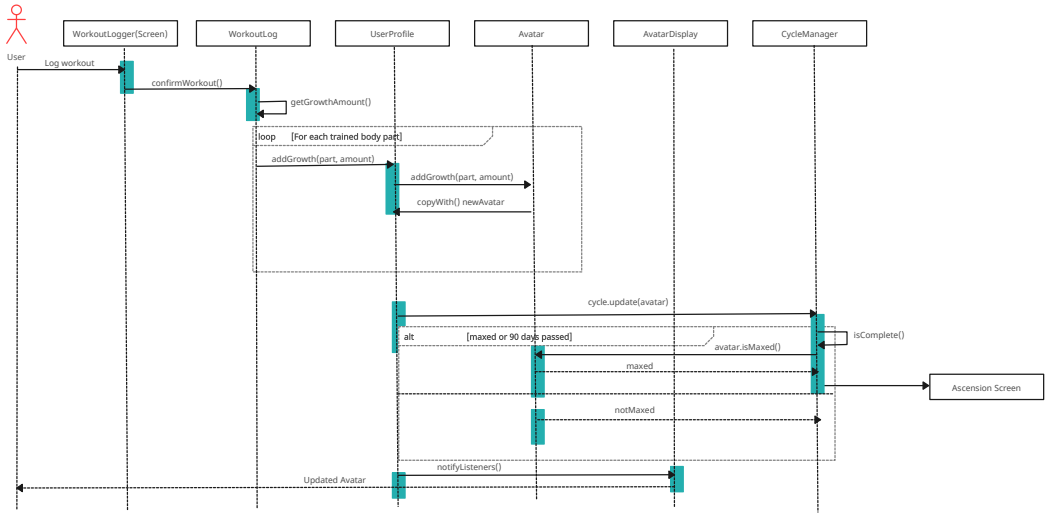


Figure 3: Sequence diagram for logging a workout in MakeMe.

The sequence begins when the user logs a workout on the workout logger screen. After confirming the workout, WorkoutLog calculates the growth amount from the duration and intensity of the session. For each body part, this amount is passed to the UserProfile, which then calls Avatar to apply the growth. Because Avatar is immutable, it returns a new Avatar with updated values. The UserProfile then completes two actions, pass the updated Avatar to the CycleManager to check if the cycle is complete and notify the listeners to update the widget AvatarDisplay to redraw the avatar. If the cycle completes the user is taken to the ascension screen and is offered to start a new cycle.

5.5. Design Patterns and Justification for Their Use

MakeMe uses three design patterns, one of them, Observer, is part of the classical list of patterns from Gang of Four book, while the other two, Immutable Object and Dependency Injection, are common practices in object-oriented programming.

- Observer: The UserProfile and ScreenManager classes extend ChangeNotifier, and the widgets that depend on them subscribe through Provider. When the data changes, the class calls notifyListeners(), and only the widgets watching it are rebuilt. MakeMe chose this pattern because the avatar needs to update on screen every time a workout is logged, and having each screen check for changes would be repetitive. With the Observer pattern, the model does not need to know which screens are displaying it.
- Immutable Object: The Avatar and HeadConfig classes are never modified in place. Methods such as copyWith(), addGrowth(), withOption() return a new object with the changed values while retaining the unchanged values. This works well alongside the Observer pattern, since a change to the avatar produces a new object, which makes it clear when the display needs to redraw. It also keeps the avatar's data in one place and reduces the chance of a program unexpectedly changing it. The trade-off is that a new object is created on every change,

but once replaced Dart's garbage collector can reclaim it. Therefore, the old copies never accumulate in memory.

- Dependency Injection: The managers are created once in app.dart via MultiProvider, and screens retrieve them with context.read instead of creating their own. This means every screen works with the same UserProfile and the same ScreenManager, so the avatar a user creates is the same one that appears on the home screen. It also keeps the screens simpler, since they do not need to know how the managers are built.

6. Implementation and Proof of Concept

6.1. Tools and Technologies

The front-end interface of MakeMe was developed using Flutter and the Dart programming language. Flutter provides a widget-based framework for building mobile application interfaces.

Android Studio and its Android emulator were used to display and inspect the application during development. Git and GitHub were used for version control and collaboration. The application also uses the Provider package for state management. A ScreenManager class, which extends ChangeNotifier, maintains the currently selected screen. When a user presses a navigation button, the setScreen() method updates the current screen and notifies the application to display the corresponding interface.

The avatar's visual assets were created in Blender [8] with MPFB2 [9], a plugin that generates human models, and Python scripts were used to automate the rendering of the avatar's sprite layers.

6.2. Implementation of the Main Features

6.2.1. Front-End Screens and Navigation

The front-end of MakeMe consists of the application's screens and the navigation between them. The following screens have been implemented.

Login Screen. The Login Screen serves as the initial entry point of the application. It includes a Start button near the bottom of the screen and a Sign Up button near the top. Pressing the Start button navigates the user to the Home Screen. The Sign Up button currently leads to a placeholder screen.

Home Screen. The Home Screen provides the main navigation interface. It contains three navigation options: Home, Loggers, and Stats. A Start Workout button is positioned near the bottom of the screen, allowing users to access the Habit Logger. The interface uses a dark blue header, a light blue background, and rounded buttons to maintain a consistent design.

Habit Logger. The Habit Logger provides four categories: Workout Log, Meal Log, Hydration Log, and Sleep Log. These options are displayed as vertically arranged buttons to make navigation simple and accessible. The Workout Log button navigates to the Workout Log Screen. The remaining logging options currently lead to placeholder screens.

Workout Log Screen. The Workout Log Screen provides an interface for accessing workout-related functionality. The current screen contains a Workout Template button that navigates to a separate Workout Template screen. A back button allows users to return to the Habit Logger.

At this stage, the Workout Log Screen primarily provides the navigation structure. Workout data entry and storage are not yet implemented in this screen.

Navigation Implementation. Screen navigation is managed through the ScreenManager class and the Screens enumeration. Each screen accesses the ScreenManager through Provider and calls setScreen() when a navigation button is pressed. This approach separates navigation state from individual screen layouts and allows the application to switch between screens without duplicating navigation logic.

6.2.2. Avatar System

The avatar system turns a user's muscle-specific progress into a visible character. The avatar is drawn as a stack of 2D sprite layers, with one layer for each body region, so that each region can change independently based on the workouts the user logs.

Asset Creation. The avatar's body was modeled in Blender using MPFB2 [8], [9], starting from MakeHuman body variants for each gender: skinny, average, heavy, and three muscle levels. These variants were combined into a single adjustable model, and its surface was divided into ten muscle regions (abs, calves, forearms, glutes, lats, pectorals, quadriceps, shoulders, trapezius, and upper arm), along with the head, hands, and feet. The nose and mouth presets used for avatar customization were created from MPFB2's built-in face shapes. Python scripts in Blender automate the process, so the full set of sprites can be regenerated whenever the model changes.

Builds and Muscle Levels. Following R1, the initial survey determines the avatar's starting build: skinny, average, athletic, or heavy. A skinny or heavy avatar transitions gradually to the average build before any muscle growth becomes visible, while an athletic avatar begins at the first muscle level in every region. Each region then grows through four levels, from 0 (average) to 3 (large). Workouts are logged by individual muscle, so the biceps and triceps are tracked separately but combined into the single upper-arm region for display.

Seamless Region Borders. Because each region is drawn as a separate layer, neighboring layers must line up exactly where they meet. Early tests that held every border at the average body shape produced visible pinching between regions as the muscles grew. To avoid this, the application tracks a shared level g, defined as the lowest level among all regions. Every border is rendered at this shared level, while each region's interior grows to its own level s. As a result, neighboring layers always align, and an avatar that grows evenly matches the original model exactly. This approach requires one sprite for each combination of shared level and region level, which is ten sprites per region and 100 muscle sprites per gender.

Rendering and Application Integration. The sprites are rendered by a Python script in Blender from a fixed orthographic camera, with the avatar in an A-pose so that the arms do not overlap the torso in the front view. Each layer is rendered with every other region masked out, in grayscale with a transparent background, at a resolution of 1200×2400 pixels. Each layer is exported at three pixel densities (600×1200, 800×1600, and 1200×2400) so that Flutter can load the version that best matches the device's screen [10]. Files follow the naming scheme `{gender}_{region}_g{g}_s{s}.png`, which allows the asset path for any combination of levels to be built in code.

In the application, the AvatarDisplay widget stacks the layers from back to front, and the AssetPaths helper builds each file path. Because the sprites are rendered in grayscale, the user's

skin tone is applied at runtime by tinting each layer with Flutter's `BlendMode.modulate`, so a single set of sprites supports any skin tone. Growth values between two levels are displayed by fading between neighboring sprites, which AvatarDisplay already supports. To complete the integration, the Avatar model must also calculate the shared level g and select each region's sprite by its pair of levels.

7. Required Resources and Feasibility

7.1. Hardware, Software, and Other Resources

The front-end development requires a computer capable of running Flutter development tools and an Android emulator. The main software resources include Flutter, Dart, Android Studio, Git, and GitHub. Flutter and Dart are used to develop the application interface, while Android Studio provides an environment for running and inspecting the application. Git and GitHub support collaborative development by allowing team members to maintain separate branches, track code changes, and integrate their contributions. The Provider package is used to manage application navigation state.

The avatar system requires additional resources to create its visual assets. Blender [8] and the MPFB2 plugin [9] are used to model the avatar and render its sprites. Both are free and open source: MPFB2 is distributed under the GPL, and MakeHuman's core assets are released under CC0 [11], so they can be used in MakeMe without licensing costs. The asset pipeline's scripts are written in Python using Blender's built-in Python API and NumPy, so no additional software is required. Rendering the sprites requires a computer capable of running Blender, as each gender needs approximately 100 muscle sprites in addition to the build and face layers, each exported at three pixel densities. Within the application, the avatar's data and progression logic are implemented in Dart in the Avatar and UserProfile classes, and the shared_preferences and sqflite packages are included in the project for storing user and avatar data locally on the device.

7.2. Feasibility Within the Available Time and Resources

The front-end implementation is feasible within the available development resources because Flutter provides reusable widgets and built-in interface components. The use of shared colors, reusable button layouts, and a centralized navigation manager reduces the complexity of implementing multiple screens. The current prototype demonstrates the initial navigation flow from the Login Screen to the Workout Log Screen. However, several features remain incomplete. Some navigation options currently lead to placeholder screens, and the Workout Log interface does not yet support recording or storing workout information. Future development will require connecting the interface to the application's data models and implementing functional input components for workout and habit tracking. Overall, the existing front-end provides a foundation for further development of MakeMe's fitness tracking functionality.

The avatar system was developed within the project's seven-week timeline. Because the avatar's visual progression depends on its assets, the asset pipeline was prioritized during the first half of this period. By the end of the third week, the male avatar model, its body regions, the build and muscle levels, the face presets, and the shared-level system had been completed.

The remaining avatar work is feasible within the available time because most of the pipeline is automated. The female avatar follows the same process, and the scripts written for the male model can be reused. All sprites are rendered by a batch script rather than by hand, so the assets can be regenerated after any change without additional manual work. In the application, the remaining tasks are extending the Avatar model with the build progress and shared level,

updating AssetPaths to the new naming scheme, and connecting AvatarDisplay to the home and avatar creation screens. Each of these tasks builds on classes that already exist.

The main risk is the number of sprites the system requires, which increases both rendering time and the size of the application. If necessary, MakeMe can reduce this by removing the highest pixel density or limiting the number of muscle levels, with only minor changes to the application. If time becomes limited, the female avatar can be completed after the male avatar is fully integrated, ensuring that the core avatar progression is demonstrated by the deadline.

8. Results and Discussion

8.1. Proposed Features and Their Completion Status

TBD

8.2. Testing and Results

TBD

8.3. Challenges, Limitations, and Reasons for Incomplete Features

TBD

9. Conclusion and Future Work

9.1. Summary of Achievements

TBD

9.2. Possible Improvements and Extensions

TBD

10. References

[1] J. Fox and J. Bailenson, "Virtual self-modeling: The effects of vicarious reinforcement and identification on exercise behaviors," *Media Psychology - MEDIA PSYCHOL*, vol. 12, pp. 1–25, Mar. 2009. DOI: 10.1080/15213260802669474

[2] A. S. Praetorius and D. Görlich, "How avatars influence user behavior: A review on the proteus effect in virtual environments and video games," in *Proceedings of the 15th international conference on the foundations of digital games*, 2020, pp. 1–9.

[3] A. Hall. "The 10 best workout and fitness apps of 2026," Forbes Health, Accessed: Oct. 12, 2026. [Online]. Available: https://www.forbes.com/health/weight-loss/best-fitness-apps/

[4] H. Herman. "The 9 best fitness apps in 2026," Zapier, Accessed: Oct. 12, 2026. [Online]. Available: https://zapier.com/blog/best-fitness-tracking-apps/

[5] A. Neupane, D. Hansen, A. Sharma, J. A. Fails, B. Neupane, and J. Beutler, "A review of gamified fitness tracker apps and future directions," in *Proceedings of the Annual Symposium on Computer-Human Interaction in Play*, 2020, pp. 522–533.

[6] G. K. Koh, K. M. Y. Low, S. H. Ali, H. S. Cheng, and I. Y. H. Ang, "Motivational factors, physical activity impacts, and sociopsychological effects of pokémon go in players over the years: Scoping review," *Journal of Medical Internet Research*, vol. 28, no. 1, e89235, 2026.

[7] J. Yim and T. N. Graham, "Using games to increase exercise motivation," in *Proceedings of the 2007 conference on Future Play*, 2007, pp. 166–173.

[8] Blender Foundation. "Blender." Version 5.2 LTS, Accessed: Nov. 10, 2026. [Online]. Available: https://www.blender.org

[9] MakeHuman Community. "MPFB: MakeHuman plugin for Blender." Software repository, Accessed: Nov. 10, 2026. [Online]. Available: https://github.com/makehumancommunity/mpfb2

[10] Google. "Adding assets and images," Flutter Documentation, Accessed: Nov. 10, 2026. [Online]. Available: https://docs.flutter.dev/ui/assets/assets-and-images

[11] MakeHuman Community. "License," Accessed: Nov. 10, 2026. [Online]. Available: https://static.makehumancommunity.org/about/license.html